Unit-5 - Question 1

Types of Errors in Python

Question 1: Explain various types of errors with examples.

1. What is an Error?

An error is a problem in a program that prevents it from running correctly. Python shows an error message and stops the program.

2. Major Types of Errors in Python

- **Syntax Error:**
  Occurs when the code does not follow Python grammar rules.

```
print("Hello"        # Missing )
```

- **Runtime Error (Exception):**
  Occurs while the program is running due to an illegal operation.

```
a = 10
b = 0
c = a / b     # ZeroDivisionError
```

- **Logical Error:**
  Program runs without any error but gives wrong output.

```
x = 5
if x > 10:
    print("Big")
else:
    print("Small")     # Output is wrong
                       # for x = 5
```

- **Indentation Error:**
  Occurs when indentation is not correct.

```
if True:
print("Yes")           # Expected an
                       # indented block
```

- **Name Error:**
  Occurs when a variable name is not defined.

```
print(y)               # NameError: name 'y'
                       # is not defined
```

- **Type Error:**
  Occurs when an operation is performed on incompatible types.

```
x = 10 + "5"           # TypeError: unsupported
                       # operand type(s)
```

- **Value Error:**
  Occurs when a function receives the correct type but an inappropriate value.

```
int("abc")             # ValueError: invalid
                       # literal for int()
```

- **Index Error:**
  Occurs when a sequence index is out of range.

```
lst = [10, 20, 30]
print(lst[5])          # IndexError: list index
                       # out of range
```

- **Key Error:**
  Occurs when a dictionary key does not exist.

```
d = {"a": 1, "b": 2}
print(d["c"])          # KeyError: 'c'
```

3. Why Do Errors Occur?

- Mistakes in writing code
- Not following syntax rules
- Wrong logic
- Forgetting indentation or names
- Wrong data or input

4. How to Reduce Errors?

- Understand Python syntax and rules.
- Test the code with different inputs.
- Use IDEs (like IDLE, VS Code, PyCharm) for hints and highlighting.
- Debug step by step and write clean, simple code.
- Use meaningful variable names.
- Read and understand error messages carefully.

5. Short Summary

Errors are of different types. Syntax and Indentation errors occur while writing code. Runtime errors (Exceptions) occur while running the program. Logical errors do not stop the program but give wrong results. Understanding and fixing errors helps in writing correct and reliable programs.

Unit-5 - Questions 2 and 3

Exception Handling and Multiple except

Question 2: Define exception? Explain different exception handling techniques.

* **Definition:** An exception is an event that occurs during the execution of a program that disrupts the normal flow of instructions. Python represents exceptions as objects and raises them when an error occurs.

* **Why Exception Handling is needed?**
  To handle run-time errors gracefully, prevent abnormal termination, maintain normal flow of the program, and provide meaningful messages to the user.

* **Basic Syntax: try - except**

```
try:
    # code that may raise exception
except ExceptionType:
    # code to handle the exception
```

```
try:
    # risky code
except ExceptionType as e:
    # handle error
else:
    # runs when no exception
```

* **Different Exception Handling Techniques:**

1) **try - except**
   Catches and handles the exception.

2) **try - except - else**
   else block runs only if no exception occurs.

3) **try - finally**
   finally block always executes whether exception occurs or not. Useful for cleanup activities.

```
try:
    # risky code
finally:
    # cleanup code (always executes)
```

4) **try - except - finally**
   Combines handling and cleanup.

```
try:
    # risky code
except ExceptionType as e:
    # handle error
finally:
    # cleanup code
```

5) **nested try**
   A try block inside another try or except block.

```
try:
    try:
        # risky code
    except ExceptionType:
        # handle inner exception
except ExceptionType:
    # handle outer exception
```

Question 3: Can we write the try with multiple except statements? Justify your answer with examples.

Answer: Yes, Python allows multiple except statements to handle different types of exceptions in different ways.

Example: Handling division by zero and value conversion errors.

```
print("Enter two numbers:")
a = input("a = ")
b = input("b = ")
try:
    x = int(a)
    y = int(b)
    z = x / y
    print("Result =", z)
except ValueError:
    print("Invalid input! Please enter integers only.")
except ZeroDivisionError:
    print("Cannot divide by zero.")
except Exception as e:
    print("An error occurred:", e)
```

Sample Outputs:

- Case 1: a = 10, b = 2
  Output: Result = 5.0
- Case 2: a = 10, b = 0
  Output: Cannot divide by zero.
- Case 3: a = abc, b = 5
  Output: Invalid input! Please enter integers only.

Flow of Control:

try block → if error occurs → match with first except that fits → execute its block → skip remaining except blocks → continue after the try statement.

* **Advantages of Exception Handling:**
  1. Prevents abnormal program termination.
  2. Helps in debugging and locating errors.
  3. Improves user experience with meaningful messages.
  4. Ensures cleanup of resources (files, connections) using finally.

* **Summary:**
  An exception is a run-time error event. Python provides several techniques such as try-except, try-except-else, try-finally, try-except-finally and nested try to handle exceptions effectively. Yes, Python allows multiple except statements, each can handle a specific type of exception differently, as shown in the example.

Unit-5 - Question 4

Built-in Exceptions in Python

10 Marks

Question 4: Explain different built-in exceptions in python with examples.

1. What are Built-in Exceptions?

- Exceptions are runtime errors that occur during the execution of a program.
- Python has a set of predefined (built-in) exceptions to handle common error situations.
- These exceptions are raised automatically by the interpreter when an error occurs.
- They can be caught and handled using try, except blocks.

2. Common Built-in Exceptions with Examples

① ZeroDivisionError : Raised when division or modulo by zero is attempted.
Example : `10 / 0` # ZeroDivisionError

② ValueError : Raised when a function receives an argument of correct type but inappropriate value.
Example : `int("abc")` # ValueError

③ TypeError : Raised when an operation or function is applied to an object of inappropriate type.
Example : `"5" + 2` # TypeError

④ NameError : Raised when a local or global name is not found.
Example : `print(x)` # NameError (x is not defined)

⑤ IndexError : Raised when a sequence subscript is out of range.
Example : `a = [10, 20, 30]; a[5]` # IndexError

⑥ KeyError : Raised when a dictionary key is not found.
Example : `d = {'a': 1, 'b': 2}; d['c']` # KeyError

⑦ FileNotFoundError : Raised when a file or path is requested but doesn't exist.
Example : `open("data.txt", "r")` # FileNotFoundError

⑧ ImportError : Raised when an import statement fails to find the module.
Example : `import not_a_module` # ImportError

⑨ AttributeError : Raised when an attribute reference or assignment fails.
Example : `"hello".append('!')` # AttributeError

⑩ IndentationError : Raised when there is incorrect indentation in code.
Example :
```
if True:
    print("Hi")
```
IndentationError

⑪ OverflowError / EOFError : OverflowError is raised when a numeric operation exceeds limits. EOFError is raised when input() hits end-of-file.
Example :
- OverflowError : `import math; math.exp(1000)` # OverflowError
- EOFError : `input()` (press Ctrl+Z then Enter) # EOFError

3. Why are Built-in Exceptions Useful?

- They help in identifying and reporting errors clearly.
- They prevent abnormal termination of the program.
- They allow developers to handle errors gracefully and write robust programs.
- They save time by providing standard exception classes for common problems.

4. Summary

Built-in exceptions in Python are predefined error types raised by the interpreter for various runtime issues. By understanding these exceptions and handling them using try-except blocks, we can make our programs more reliable, readable and user-friendly.

--- End ---

Unit-5 - Question 5

User-Defined Exceptions

Question 5: How to create and raise the user defined exceptions with an examples.

1. **Meaning:**
   - User-defined exceptions are custom exceptions created by the programmer to represent application specific error conditions.
2. **Why we create custom exceptions?**
   - To handle application specific errors in a meaningful way.
   - To provide clear and descriptive error messages.
   - To improve code readability and maintainability.
   - To separate business logic errors from built-in Python errors.
3. **Steps to create a user-defined exception (by inheriting from Exception):**
   1. Create a class and inherit from Exception.
   2. (Optional) Define a constructor to customize the error message.
   3. Use the raise keyword to raise the exception.
   4. Handle it using try-except block.
4. **How to raise a user-defined exception:**
   - Use the raise keyword followed by an instance of your custom exception class.
   - You can pass a custom error message to provide more information.
5. **Example 1: AgeError**

```
class AgeError(Exception):
    def __init__(self, message):
        super().__init__(message)
```

```
def check_age(age):
    if age < 0:
        raise AgeError("Age cannot be negative.")
    if age < 18:
        raise AgeError("Age must be at least 18.")
    return "Access granted!"
```

```
try:
    age = int(input("Enter your age: "))
    result = check_age(age)
    print(result)
except AgeError as e:
    print("AgeError:", e)
```

[If user enters -5 → "Age cannot be negative."
If user enters 15 → "Age must be at least 18."
If user enters 20 → "Access granted!"]

6. **Example 2: InvalidMarksError**

```
class InvalidMarksError(Exception):
    def __init__(self, message):
        super().__init__(message)
```

```
def validate_marks(marks):
    if not (0 <= marks <= 100):
        raise InvalidMarksError("Marks must be between 0 and 100.")
    return "Valid marks"
```

```
try:
    m = int(input("Enter marks: "))
    print(validate_marks(m))
except InvalidMarksError as e:
    print("InvalidMarksError:", e)
```

7. **Advantages:**
   - Makes error handling more specific.
   - Provides meaningful and user-friendly messages.
   - Improves code structure and maintainability.
   - Easier debugging and understanding.
8. **Built-in vs User-Defined Exceptions**

Built-in Exceptions	User-Defined Exceptions
Provided by Python.	Created by the programmer.
Handle general Python errors.	Handle application specific errors.
Examples: ValueError, TypeError, FileNotFoundError, etc.	Examples: AgeError, InvalidMarksError, etc.
Fixed set of exceptions.	Unlimited, as per requirement.

9. **Summary:**

User-defined exceptions allow programmers to define and raise custom exceptions for application specific error conditions. By inheriting from Exception and using the raise keyword, we can throw meaningful errors and handle them using try-except for better control and clarity in our programs.

Unit-5 – Question 6

Clean-up Activities in Python

Question 6: Explain different clean-up activities with suitable examples.

[10 Marks]

1. What are clean-up activities?

Clean-up activities are the tasks performed to release resources and restore the system to a proper state after the completion of a task or when an exception occurs.

2. Importance of releasing resources

- Prevents resource leaks (memory, file handles, connections).
- Ensures data is saved and not lost.
- Allows other programs/users to use the resources.
- Keeps the application stable and reliable.

3. Different clean-up activities

① Use finally block – Code inside finally executes no matter what (normal execution or exception).

② Close files using close() – Always close files after reading/writing.

③ Automatic clean-up with with open(...) – The with statement automatically closes the file.

④ Delete or reset variables/objects – Remove large objects or reset variables to free memory if needed (e.g., del obj or obj = None).

⑤ Close database/network connections – Always close DB connections, cursors, sockets, or network connections.

⑥ Other general clean-up – Remove temporary files, release locks, stop background threads, etc.

4. Example 1: Using finally with file handling

```
def read_file(fname):
    f = None
    try:
        f = open(fname, 'r')
        data = f.read()
        print("File content:", data)
    except FileNotFoundError:
        print("File not found!")
    finally:
        if f:
            f.close()
            print("File closed in finally block.")
```

Here, f.close() in finally executes whether the file is read successfully or an exception occurs.

5. Example 2: Automatic clean-up with with statement

```
try:
    with open('data.txt', 'r') as f:
        data = f.read()
        print("File content:", data)
except FileNotFoundError:
    print("File not found!")
```

When the block inside with is exited, the file is automatically closed. No need to call close().

6. Summary / Benefits

- Clean-up activities help in proper resource management.
- finally ensures important code runs always.
- with statement simplifies clean-up and makes code safer and shorter.

In short, always clean-up resources properly to avoid leaks, ensure data safety and make programs reliable.

Unit-5 - Questions 7 and 8

GUI in Python and Common Widgets

Question 7: Explain how to create the GUI application in python with examples?

- GUI (Graphical User Interface) allows users to interact with an application using windows, buttons, menus, icons, etc. instead of typing text commands.
- Difference (GUI vs Text Interface):

GUI (Graphical)	Text / Command Line (CLI)
Uses mouse and visual components.	Uses keyboard and typed commands.
Easy to learn and use.	Requires command knowledge.
Good for general users.	Good for advanced users and scripting.
Example: Tkinter, PyQt	Example: cmd, terminal

- Tkinter is the built-in GUI library in Python. It is simple and easy to use.
- Steps to create a GUI application using Tkinter:

① Import tkinter module.
② Create the root window (main window).
③ Set window title and size.
④ Add widgets (buttons, labels, text boxes, etc.).
⑤ Start the event loop using mainloop().

- Example 1: A simple "Hello Window"

```
import tkinter as tk
# 1. Create root window
root = tk.Tk()
# 2. Set title and size
root.title("Hello GUI")
root.geometry("300x150")
# 3. Add a label widget
label = tk.Label(root, text="Hello, Welcome to Tkinter GUI!")
label.pack(pady=20)
# 4 & 5. Start the event loop
root.mainloop()
```



Question 8: Explain different widgets used in GUI based applications with examples.

	Widget	Use / Description	Example (very small)
①	Label	Displays text or image.	`tk.Label(root, text="Name:").pack()`
②	Button	Performs an action when clicked.	`tk.Button(root, text="Click", command=func).pack()`
③	Entry	Single line text input from user.	`e = tk.Entry(root); e.pack()`
④	Text	Multi-line text input or display.	`t = tk.Text(root, height=4, width=20); t.pack()`
⑤	Frame	Container to group widgets.	`f = tk.Frame(root, bd=2, relief="sunken"); f.pack()`
⑥	Checkbutton	Select multiple options (on/off).	`tk.Checkbutton(root, text="Python").pack()`
⑦	Radiobutton	Select one option from many.	`tk.Radiobutton(root, text="Male", value=1).pack()`
⑧	Listbox	Shows a list of items to select.	`lb = tk.Listbox(root); lb.insert(tk.END, "A"); lb.pack()`
⑨	Menu	Menu bar with dropdown items.	`menubar = tk.Menu(root); file = tk.Menu(menubar)` `menubar.add_cascade(label="File", menu=file)`
⑩	Messagebox	Shows alert, info, error dialogs.	`from tkinter import messagebox` `messagebox.showinfo("Info", "Operation Successful")`

Advantages of GUI:

- User friendly and easy to interact.
- Improves productivity and reduces errors.
- Good presentation and better user experience.
- Suitable for most desktop applications.

Summary:

Tkinter helps us create windows and add widgets like Label, Button, Entry, etc. We design the interface and start the mainloop to run the app. GUI makes applications attractive and easy to use.

— All the Best! ☺ —

Unit-5 - Question 9

Terminal-based vs GUI-based Programming

Question 9: Write the differences between terminal based and GUI based programming.

Terminal-based Programming: Applications that interact with the user through a text-based interface (command line/terminal). Users type commands and view text output.

GUI-based Programming: Applications that interact with the user through graphical elements such as windows, buttons, menus, icons, and dialogs. Users interact using mouse, keyboard, and other input devices.

Aspect	Terminal-based Programming	GUI-based Programming
1. Interaction method	User types commands in a terminal.	User interacts using mouse, keyboard, touch, etc. on graphical elements.
2. Input/Output style	Text-based input and text output.	Graphical input (clicks, selections, drag & drop) and rich visual output.
3. Ease of use	Requires knowledge of commands and syntax.	Easy to learn and operate for most users.
4. Speed	Generally faster for experienced users (keyboard shortcuts, scripts).	Relatively slower due to rendering graphics and events.
5. Appearance	Simple, plain text interface.	Rich, colorful, and visually appealing interface.
6. User friendliness	Less user-friendly for beginners.	More user-friendly and intuitive.
7. Hardware/Resource usage	Consumes fewer system resources; works well on low-end systems.	Consumes more memory and CPU due to graphics and libraries.
8. Examples	Command Prompt (cmd), Bash, PowerShell, Python shell.	Windows Apps, Microsoft Word, Web browsers, IDEs like PyCharm, VS Code (GUI), etc.
9. Suitability	Best for system administration, automation, scripting, programming, and server environments.	Best for general users, desktop applications, games, and tasks requiring visual interaction.
10. Learning difficulty	Steeper learning curve for new users.	Easier to learn and adopt.

Summary:

- → Terminal-based programming is text-oriented, fast, lightweight, and powerful, but less intuitive for beginners.
- → GUI-based programming is visual, interactive, and user-friendly, but requires more resources and may be slower.

Both approaches have their own strengths and are used based on the application requirements and the target users.

Unit-5 - Question 10

Important Programs - Page 1

Exception Handling Programs

1) Handle division by zero using try-except.

```
a = 10
b = 0
try:
    c = a / b
    print("Result =", c)
except ZeroDivisionError:
    print("Error: Cannot divide by zero.")
```

Output:

```
Error: Cannot divide by zero.
```

---

2) Convert string to integer with ValueError handling.

```
s = input("Enter a number: ")
try:
    n = int(s)
    print("Integer =", n)
except ValueError:
    print("Error: Invalid input. Please enter a
          valid integer.")
```

Output (if input = '25'):

```
Enter a number: 25
Integer = 25
```

Output (if input = 'abc'):

```
Enter a number: abc
Error: Invalid input.
  Please enter a valid integer.
```

---

3) Use multiple except blocks for ZeroDivisionError and ValueError.

```
a = input("Enter first number: ")
b = input("Enter second number: ")
try:
    result = int(a) / int(b)
    print("Result =", result)
except ZeroDivisionError:
    print("Error: Division by zero.")
except ValueError:
    print("Error: Please enter valid numbers.")
```

Output examples:

Case 1: a = 10, b = 0

```
Error: Division by zero.
```

Case 2: a = 'ten', b = 2

```
Error: Please enter valid numbers.
```

Case 3: a = 10, b = 2

```
Result = 5.0
```

---

4) Use finally block to show clean-up message.

```
try:
    x = int(input("Enter a number: "))
    print("Double =", 2 * x)
except ValueError:
    print("Invalid number!")
finally:
    print("Clean up: Thank you!")
```

Output (if input = '7'):

```
Enter a number: 7
Double = 14
Clean up: Thank you!
```

Output (if input = 'abc'):

```
Enter a number: abc
Invalid number!
Clean up: Thank you!
```

---

5) Demonstrate FileNotFoundError handling while opening a file.

```
try:
    f = open("sample.txt", "r")
    print("File opened successfully!")
    f.close()
except FileNotFoundError:
    print("Error: File not found.")
```

Output (if file exists):

```
File opened successfully!
```

Output (if file does not exist):

```
Error: File not found.
```

Unit-5 - Question 10

Important Programs - Page 2

More Exception and Custom Exception Programs

6) Create and raise a user-defined exception (AgeError)

```
class AgeError(Exception):
    pass
try:
    age = int(input("Enter age: "))
    if age < 0 or age > 120:
        raise AgeError("Age must be between 0 and 120.")
    print("Age is valid.")
except AgeError as e:
    print("Error:", e)
```

Output:

```
Enter age: -5
Error: Age must be
    between 0 and 120.
```

7) Nested try example

```
try:
    a = int(input("Enter a: "))
    try:
        b = int(input("Enter b: "))
        c = a / b
        print("Result =", c)
    except ZeroDivisionError:
        print("Inner Exception: Division by zero")
except ValueError:
    print("Outer Exception: Invalid number")
```

Output 1 (b = 0):

```
Enter a: 10
Enter b: 0
Inner Exception: Division by zero
```

Output 2 (a = x):

```
Enter a: x
Outer Exception: Invalid number
```

8) Else block with try-except example

```
try:
    n = int(input("Enter a positive number: "))
    print("Square =", n*n)
except ValueError:
    print("Please enter a valid number.")
else:
    print("No exception occurred.")
finally:
    print("Finally block always executes.")
```

Output (input 5):

```
Enter a positive number: 5
Square = 25
No exception occurred.
Finally block always executes.
```

Output (input x):

```
Enter a positive number: x
Please enter a valid number.
Finally block always executes.
```

9) Program to count lines/words with exception handling for missing file

```
try:
    filename = input("Enter file name: ")
    f = open(filename, 'r')
    lines = f.readlines()
    words = sum(len(line.split()) for line in lines)
    print("Lines:", len(lines))
    print("Words:", words)
    f.close()
except FileNotFoundError:
    print("Error: File not found.")
```

Output (file exists):

```
Enter file name: sample.txt
Lines: 5
Words: 38
```

Output (file missing):

```
Enter file name: data.txt
Error: File not found.
```

10) Use with open() and exception handling together

```
try:
    with open('sample.txt', 'r') as f:
        data = f.read()
        print("File content:\n", data)
except FileNotFoundError:
    print("File not found.")
except IOError as e:
    print("I/O error:", e)
```

Output (file exists):

```
File content:
Hello Python!
Exception handling is useful.
```

Output (file missing):

```
File not found.
```

Unit-5 - Question 10

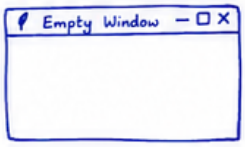
Important Programs - Page 3

GUI Programs using Tkinter

11) Create a basic empty window.

```
import tkinter as tk
root = tk.Tk()
root.title("Empty Window")
root.geometry("300x200")
root.mainloop()
```

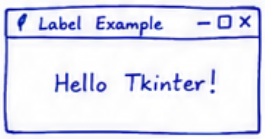
Output / Use : Opens a blank window.



12) Display a Label in a window.

```
import tkinter as tk
root = tk.Tk()
root.title("Label Example")
lbl = tk.Label(root, text="Hello Tkinter!",
               font=("Arial", 16))
lbl.pack(padx=20, pady=20)
root.mainloop()
```

Output / Use : Shows a text label.



13) Button click showing a message or changing label text.

```
import tkinter as tk
from tkinter import messagebox
root = tk.Tk()
root.title("Button Example")
lbl = tk.Label(root, text="Click the button")
lbl.pack(pady=10)

def on_click():
    lbl.config(text="Button Clicked!")
    messagebox.showinfo("Info", "Button was clicked!")

btn = tk.Button(root, text="Click Me", command=on_click)
btn.pack(pady=10)
root.mainloop()
```

Output / Use : Click button → label changes and message box appears.

14) Simple addition calculator using two Entry widgets and a Button.

```
import tkinter as tk
root = tk.Tk()
root.title("Addition Calculator")
tk.Label(root, text="Enter first number:").grid(row=0, column=0, padx=5, pady=5)
e1 = tk.Entry(root); e1.grid(row=0, column=1, padx=5, pady=5)
tk.Label(root, text="Enter second number:").grid(row=1, column=0, padx=5, pady=5)
e2 = tk.Entry(root); e2.grid(row=1, column=1, padx=5, pady=5)

def add():
    try:
        result = float(e1.get()) + float(e2.get())
        res.set(str(result))
    except:
        res.set("Invalid input")

btn = tk.Button(root, text="Add", command=add)
btn.grid(row=2, column=0, padx=5, pady=10)
res = tk.StringVar()
tk.Label(root, textvariable=res, font=("Arial", 12)).grid(row=2, column=1)
root.mainloop()
```

Use : Enters two numbers and shows the sum.

15) Checkbutton and Radiobutton example.

```
import tkinter as tk
root = tk.Tk()
root.title("Options Example")

# Checkbuttons
v1 = tk.IntVar(); v2 = tk.IntVar()
tk.Checkbutton(root, text="Python", variable=v1).pack(anchor='w')
tk.Checkbutton(root, text="Java", variable=v2).pack(anchor='w')

tk.Label(root, text="Choose your course:").pack(pady=(10,0))
choice = tk.StringVar(value="Python")
tk.Radiobutton(root, text="Python",
               variable=choice, value="Python").pack(anchor='w')
tk.Radiobutton(root, text="Java",
               variable=choice, value="Java").pack(anchor='w')

root.mainloop()
```

Use : Check one or more options (Checkbutton)
Select one option (Radiobutton).

16) Listbox example.

```
import tkinter as tk
root = tk.Tk()
root.title("Listbox Example")

lb = tk.Listbox(root, height=5)
for item in ["Apple", "Banana", "Mango", "Orange", "Grapes"]:
    lb.insert(tk.END, item)
lb.pack(padx=10, pady=10)

root.mainloop()
```

Output / Use : Displays a list of items.



Note : Run each program separately in a Python environment that supports Tkinter. ☺